

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

区间 DP：从石子合并到回文串

Interval Dynamic Programming: From Stone Merging to Palindromes

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. 区间 DP 概述
2. 石子合并
3. 最长回文子序列
4. 括号匹配
5. 经典例题
6. 总结



目录

1 区间 DP 概述

2 石子合并

3 最长回文子序列

4 括号匹配

5 经典例题

6 总结

什么是区间 DP

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

区间 DP：状态由一个区间 $[l,\ r]$ 描述，答案由更小的子区间组合而来。

两个关键特征

- **状态是区间**：$dp[l][r]$ 表示区间 $[l,\ r]$ 上的最优解
- **转移是划分**：枚举一个分割点 k，把 $[l,\ r]$ 拆成 $[l,\ k]$ 和 $[k+1,\ r]$

常见适用场景

- 区间合并类：石子合并、能量项链
- 区间匹配类：括号匹配、最长回文子序列
- 区间划分类：多边形三角剖分、矩阵链乘

区间 DP 的核心：大区间的最优解，来自小区间最优解的合并

区间 DP 的三层循环

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

一般骨架

- 第一层：枚举**区间长度** len，从 1 到 n
- 第二层：枚举区间**左端点** l，右端点 $r = l + len - 1$
- 第三层：枚举**分割点** k，从 l 到 $r - 1$

为什么长度放最外层

- $dp[l][r]$ 依赖区间更短的状态
- 按长度从小到大推，保证依赖项已经算好
- 不能按 l 或 r 直接顺序推，会读到未计算的状态

区间 DP 的复杂度通常是 $O(n^3)$：长度、左端、分割点各一层

区间 DP 的通用转移

模板形式

$$dp[l][r] = \min_{l \le k < r} \{dp[l][k] + dp[k+1][r] + cost(l, r)\}$$

三个组成部分

- $dp[l][k]$：左子区间的答案
- $dp[k+1][r]$：右子区间的答案
- $cost(l, r)$：把两半合并起来的额外代价

常见变形

- 取 $\min$ 或 $\max$，取决于题目要求
- 匹配类问题不一定用分割点，而是看两端字符是否匹配

划分型用分割点 k；匹配型看两端 l 与 r

目录

1 区间 DP 概述

2 石子合并

3 最长回文子序列

4 括号匹配

5 经典例题

6 总结

例题一：石子合并

题目：有 n 堆石子排成一排，第 i 堆有 $a[i]$ 个。每次可以合并**相邻**的两堆，合并的代价是两堆石子数之和。合并后新堆的石子数是两堆之和。求把所有石子合并成一堆的**最小总代价**。

示例

- $n = 4$，**石子数** $a = [1,\ 3,\ 5,\ 2]$
- **一种方案**：**合并** $1 + 3 = 4$（**代价** 4），**再合并** $4 + 5 = 9$（**代价** 9），**最后** $9 + 2 = 11$（**代价** 11），**总代价** $4 + 9 + 11 = 24$
- **最优方案**：**合并** $5 + 2 = 7$（**代价** 7），**合并** $1 + 3 = 4$（**代价** 4），**合并** $4 + 7 = 11$（**代价** 11），**总代价** $7 + 4 + 11 = 22$

石子合并：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[l][r]$ 表示把区间 $[l,\ r]$ 合并成一堆的最小代价
- 转移：枚举最后一次合并的分割点 k
- 方程：$dp[l][r] = \min_{l \le k < r}\{dp[l][k] + dp[k+1][r]\} + \text{sum}(l, r)$
- 边界：$dp[i][i] = 0$，单独一堆不需要合并

为什么加 $\text{sum}(l, r)$

- 最后一步合并把两个子堆并成一堆
- 合并的代价就是区间 $[l,\ r]$ 上所有石子的和
- 用前缀和 $O(1)$ 求出

枚举最后一次合并的位置，把问题拆成两个独立子问题

石子合并：代码


```cpp
// dp[l][r] 表示把 a[l..r] 合并成一堆的最小代价
int stoneMerge(vector<int>& a) {
    int n = a.size();
    vector<int> pre(n + 1, 0);                  // 前缀和
    for (int i = 1; i <= n; i++) pre[i] = pre[i-1] + a[i-1];

    const int INF = 1e9;
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= n; i++) dp[i][i] = 0;

    // 枚举区间长度，从 2 开始 *
    for (int len = 2; len <= n; len++) {
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            dp[l][r] = INF;
            for (int k = l; k < r; k++) {   // 枚举分割点
                dp[l][r] = min(dp[l][r],
                               dp[l][k] + dp[k+1][r]);
```

石子合并：代码

```
19              }
20              dp[l][r] += pre[r] - pre[l-1];  // 加上区间和 *
21          }
22      }
23      return dp[1][n];
24  }
```

复杂度 $O(n^3)$，**空间** $O(n^2)$。

石子合并：环形扩展

题目变体：石子排成一个环，首尾相邻，求合并成一堆的最小代价。

思路：破环成链

- 把原数组复制一份拼到后面：$a[1..2n] = a[1..n] + a[1..n]$
- 在长度 $2n$ 的链上做区间 DP
- 答案是 $\min_{1 \le i \le n} dp[i][i+n-1]$

为什么可以

- 环上任取一个起点，展开就是链
- 枚举所有可能的起点，取最小
- 答案在长度 n 的窗口里

环形区间 DP 的通用套路：数组翻倍，窗口长度保持 n

目录

1 区间 DP 概述

2 石子合并

3 最长回文子序列

4 括号匹配

5 经典例题

6 总结

例题二：最长回文子序列

题目：给定字符串 s，求它的最长回文子序列长度。子序列不要求连续，但要保持相对顺序。

示例

- $s =$ bbbab，最长回文子序列是 bbbb，长度为 4
- $s =$ cbbd，最长回文子序列是 bb，长度为 2

提醒

- 子序列：可以跳过字符
- 子串：必须连续
- 本题是子序列，不要求连续

最长回文子序列：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[l][r]$ 表示 $s[l..r]$ 的最长回文子序列长度
- 转移：看 $s[l]$ 与 $s[r]$ 是否相等
- 若相等：$dp[l][r] = dp[l+1][r-1] + 2$
- 否则：$dp[l][r] = \max(dp[l+1][r],\ dp[l][r-1])$
- 边界：$dp[i][i] = 1$，单字符长度 1

为什么是这种转移

- 两端相同：两端可同时纳入回文，问题缩小到内部
- 两端不同：至少有一个不进回文，取左右各自延伸的较大者

匹配型区间 DP：看两端是否相等，而不是枚举分割点

最长回文子序列：代码


```cpp
// dp[l][r] 表示 s[l..r] 的最长回文子序列长度
int longestPalindromeSubseq(string s) {
    int n = s.size();
    if (n == 0) return 0;
    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) dp[i][i] = 1;   // 单字符长度 1 *

    // 按区间长度从小到大推 *
    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;
            if (s[l] == s[r])
                dp[l][r] = dp[l+1][r-1] + 2;    // 两端相同，纳入回文 *
            else
                dp[l][r] = max(dp[l+1][r], dp[l][r-1]);   // 否则取较大者 *
        }
    }
```

最长回文子序列：代码


```
19      return dp[0][n-1];
20  }
```

答案在 $dp[0][n-1]$，即整个串上的最长回文子序列长度。

目录

1 区间 DP 概述

2 石子合并

3 最长回文子序列

4 括号匹配

5 经典例题

6 总结



例题三：最长有效括号

题目：给定一个只含（ 和 ）的字符串，求最长的、左右括号匹配的连续子串长度。

示例

- $s =$ (()，最长有效括号是 ()，长度为 2
- $s =$)()())，最长有效括号是 ()()，长度为 4
- $s =$ ()(())，最长有效括号是 ()(())，长度为 6

最长有效括号：解法

四步法

- 状态：$dp[i]$ 表示以 $s[i]$ **结尾**的最长有效括号长度
- 转移：只考虑 $s[i] =$ ） 的情况（以 ）结尾才可能匹配）
- 若 $s[i-1] =$ （ ：$dp[i] = dp[i-2] + 2$
- 若 $s[i-1] =$ ） 且 $s[i - dp[i-1] - 1] =$ （ ：
  $dp[i] = dp[i-1] + 2 + dp[i - dp[i-1] - 2]$
- 边界：$dp[0] = 0$，答案取 $\max_i dp[i]$

两种匹配情况

- 前一位是（ ：直接配成一对，再拼上前面一段有效括号
- 前一位是 ）：跳过前一段有效括号，看它前面是不是（

“以 i 结尾” + 往前跳 dp 长度，是匹配类问题的常用套路

最长有效括号：代码


```cpp
// dp[i] 表示以 s[i] 结尾的最长有效括号长度
int longestValidParentheses(string s) {
    int n = s.size();
    vector<int> dp(n, 0);
    int ans = 0;

    for (int i = 1; i < n; i++) {
        if (s[i] != ')') continue;              // 只处理右括号 *
        if (s[i-1] == '(') {
            // 前一位是左括号，直接配对 *
            dp[i] = 2 + (i >= 2 ? dp[i-2] : 0);
        } else if (dp[i-1] > 0) {
            // 前一位是右括号，跳过前一段有效括号 *
            int j = i - dp[i-1] - 1;
            if (j >= 0 && s[j] == '(') {
                dp[i] = dp[i-1] + 2;
                if (j >= 1) dp[i] += dp[j-1];
            }
```

最长有效括号：代码


```
19          }
20          ans = max(ans, dp[i]);
21      }
22      return ans;
23  }
```

目录

1 区间 DP 概述

2 石子合并

3 最长回文子序列

4 括号匹配

5 经典例题

6 总结



例题四：多边形三角剖分

题目：给定一个凸多边形，每个顶点有一个权值 $v[i]$。用若干条不相交的对角线把多边形剖分成若干个三角形，每次剖分的代价是三个顶点权值的乘积。求最小总代价。

示例

- 三角形 $(n = 3)$：$v = [1,\ 2,\ 3]$，唯一剖分，代价 $1 \times 2 \times 3 = 6$
- 四边形 $(n = 4)$：$v = [1,\ 2,\ 3,\ 4]$，有两种剖分，取较小的代价

多边形三角剖分：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[l][r]$ 表示顶点 l 到 r 构成的子多边形的最小剖分代价
- 转移：枚举分割点 k，把子多边形分成 $[l,\ k]$ 和 $[k,\ r]$
- 方程：$dp[l][r] = \min\{dp[l][k] + dp[k][r] + v[l] \times v[k] \times v[r]\}$
- 边界：$dp[i][i+1] = 0$，两点构不成三角形

与石子合并的对比

- 状态都是区间 $[l,\ r]$，但这里 l 和 r 是共享顶点
- 分割点 k 在 l 与 r 之间，形成两个共享一条边的子多边形
- 代价函数换成三顶点权值乘积

区间 DP 的模板结构不变，只是代价函数换了

多边形三角剖分：代码


```cpp
// dp[l][r] 表示顶点 l 到 r 构成子多边形的最小代价
int minScoreTriangulation(vector<int>& v) {
    int n = v.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));

    // 按子多边形顶点数从小到大推 *
    for (int len = 3; len <= n; len++) {
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;
            dp[l][r] = INT_MAX;
            // 枚举分割点 k，把多边形分成两个子多边形 *
            for (int k = l + 1; k < r; k++) {
                dp[l][r] = min(dp[l][r],
                               dp[l][k] + dp[k][r] + v[l] * v[k] * v[r]);
            }
        }
    }
    return dp[0][n-1];
```

多边形三角剖分：代码


```
19 }
```

注意 $dp[l][k]$ 与 $dp[k][r]$ 共享顶点 k，这是与石子合并的主要区别。

例题五：戳气球

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

题目：有 n 个气球排成一排，第 i 个气球上写有数字 $nums[i]$。戳破气球 i 可以得到 $nums[i-1] \times nums[i] \times nums[i+1]$ 个硬币（相邻气球为边界时按 1 计）。求能获得的最大硬币数。

示例

- $nums = [3,\ 1,\ 5,\ 8]$
- **一种最优方案**：先戳 1（得 $3 \times 1 \times 5 = 15$），再戳 5（得 $3 \times 5 \times 8 = 120$），再戳 3（得 $1 \times 3 \times 8 = 24$），最后戳 8（得 $1 \times 8 \times 1 = 8$），总得 $15 + 120 + 24 + 8 = 167$

戳气球：解法

关键思路：反向思考

- 正着想“先戳谁”会互相影响，很难拆分
- 反着想“**最后戳谁**”：最后一个戳的气球把问题分成左右两部分

四步法

- 状态：$dp[l][r]$ 表示戳破开区间 $(l,\ r)$ 内所有气球的最大收益
- 转移：枚举区间内**最后一个戳破**的气球 k
- 方程：$dp[l][r] = \max\{dp[l][k] + dp[k][r] + nums[l] \times nums[k] \times nums[r]\}$
- 边界：在数组两端各加一个虚拟气球，值为 1

“最后一步”视角是区间 DP 的常见思维转向

戳气球：代码


```cpp
// 在数组两端补 1，方便处理边界 *
int maxCoins(vector<int>& nums) {
    int n = nums.size();
    vector<int> a(n + 2, 1);
    for (int i = 0; i < n; i++) a[i+1] = nums[i];

    int m = n + 2;
    vector<vector<int>> dp(m, vector<int>(m, 0));

    // dp[l][r] 表示戳破开区间 (l, r) 内所有气球的最大收益 *
    for (int len = 2; len <= m - 1; len++) {
        for (int l = 0; l + len < m; l++) {
            int r = l + len;
            for (int k = l + 1; k < r; k++) {
                // k 是最后戳破的气球 *
                int gain = a[l] * a[k] * a[r];
                dp[l][r] = max(dp[l][r],
                               dp[l][k] + dp[k][r] + gain);
```

戳气球：代码


```
19                 }
20             }
21         }
22         return dp[0][m-1];
23     }
```

两端补 1 是常用技巧，让“边界”的乘积计算统一。

目录

1 区间 DP 概述

2 石子合并

3 最长回文子序列

4 括号匹配

5 经典例题

6 总结

区间 DP 的核心结构

- 状态：$dp[l][r]$，表示区间 $[l,\ r]$ 上的最优解
- 转移：两种典型模式
  - **划分型**：枚举分割点 k，把区间拆成两半
  - **匹配型**：看两端 l 与 r 是否匹配，向内收缩
- 边界：长度为 1 或 2 的区间单独处理
- 顺序：按区间长度从小到大推

先定状态看区间，再想转移是划分还是匹配，最后确定长度顺序

经典题型速查

题型	转移类型	转移核心
石子合并	划分型	$dp[l][k] + dp[k+1][r] +$ sum
最长回文子序列	匹配型	两端相等加 2，否则取 $\max$
最长有效括号	匹配型	看前一位，往前跳 dp 长度
多边形三角剖分	划分型	$dp[l][k] + dp[k][r] + v_l v_k v_r$
戳气球	划分型（反向）	枚举最后戳破的气球

划分型枚举分割点；匹配型看两端；反向题找“最后一步”

常见易错点

- 循环顺序错：按 l 或 r 顺序推，读到未计算的状态
- 区间定义含糊：$dp[l][r]$ 是闭区间还是开区间，全程要统一
- 分割点范围错：划分型 k 从 l 到 $r-1$，不要越界
- 边界忘记初始化：长度为 1 或 2 的区间要单独设初值
- 环形问题忘记破环成链：翻倍数组但窗口长度保持 n
- 匹配型忘记空区间：$l > r$ 时 dp 应为 0，不是默认值
- 代价函数漏乘：如三角剖分忘乘 $v_l \times v_k \times v_r$

下一步学什么

- 熟练写出划分型与匹配型两种区间 DP 模板
- 掌握环形区间 DP 的翻倍套路
- 学会从“最后一步”反推转移方程
- 为后续学习更复杂的 DP 打基础：
  - 四边形不等式优化（$O(n^3) \to O(n^2)$）
  - 区间 DP 与博弈结合（取数游戏）
  - 树形 DP 与区间 DP 的相互转化

区间 DP 的关键：想清楚区间怎么拆，以及代价怎么加

附录：划分型 vs 匹配型

对比项	划分型	匹配型
典型问题	石子合并、三角剖分	回文、括号
转移方式	枚举分割点 k	看两端 l 与 r
第三层循环	有，$O(n)$	无
转移代价	加上 $cost(l, r)$	加 2 或取 $\max$

识别题型的关键：题目是否明显需要“切开”

附录：环形区间 DP 套路

题目特征：首尾相邻，形成环。

通用解法

1. 把数组复制一份拼到后面，得到长度 $2n$ 的链
2. 在长度 $2n$ 的链上做区间 DP
3. 答案是 $\min$ 或 $\max$ 长度 n 的所有窗口
4. 即 $\min / \max\{dp[i][i+n-1]\}$，$1 \le i \le n$

典型题目

- 环形石子合并
- 能量项链
- 环形矩阵链乘

环形问题的核心：破环成链，多花一倍空间换思路简洁

附录：复杂度与优化

问题	时间复杂度	空间复杂度
普通区间 DP	$O(n^3)$	$O(n^2)$
环形（翻倍）	$O((2n)^3)$	$O((2n)^2)$
四边形不等式优化	$O(n^2)$	$O(n^2)$
一维滚动优化	不变	$O(n)$（部分场景）

入门先把 $O(n^3)$ 写熟；优化是后续进阶话题

附录：区间 DP 通用模板

```cpp
// 划分型模板
for (int len = 2; len <= n; len++) {
    for (int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;                    // 或 -INF
        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r],
                           dp[l][k] + dp[k+1][r] + cost(l, r));
        }
    }
}

// 匹配型模板
for (int len = 2; len <= n; len++) {
    for (int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        if (match(l, r))
            dp[l][r] = dp[l+1][r-1] + 2;
```

附录：区间 DP 通用模板

```
19          else
20              dp[l][r] = max(dp[l+1][r], dp[l][r-1]);
21      }
22  }
```

两套模板覆盖绝大多数区间 DP 题目；先把它们敲熟，再考虑优化。